

Introducción Desarrollo WEB

1. Introducción

En este tema abordaremos conceptos generales relacionados con las aplicaciones web:

- Arquitectura Cliente/Servidor
- Lenguajes utilizados
- Frameworks disponibles

1.1. La base de la web

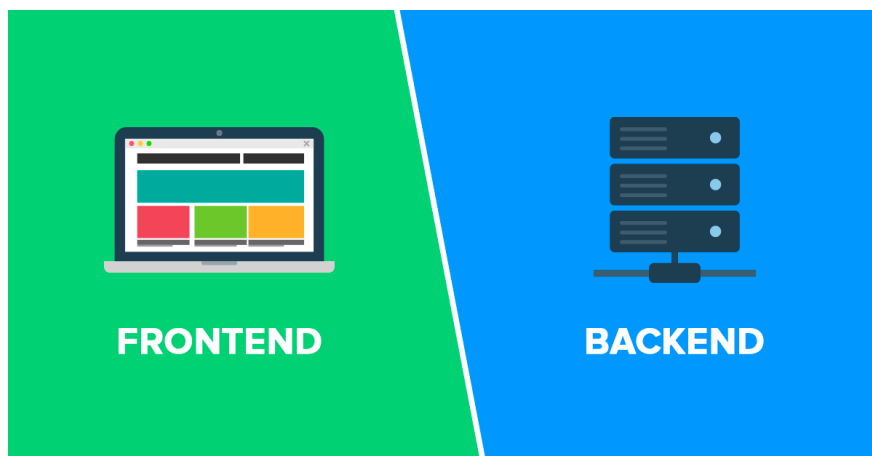


- **HTML:** Estructura del documento. HTML5. 2014
- **CSS:** Formato/apariencia del documento. CSS3.
- **Javascript:** Funcionalidad del documento. ECMAScript6. 2015

Note

Al final de línea se muestra la versión más relevante actualmente y el año de su aparición oficial.

1.2. Partes de una aplicación web



- **Backend:** parte que se ejecuta en el servidor.
 - Se encarga de atender las peticiones de los clientes.
 - Suele tener soporte de almacenamiento de datos.
 - También se denomina capa de acceso a datos
- **Frontend:** parte que se ejecuta en el cliente.
 - Se encarga de la experiencia del usuario (UX).
 - Puede tener soporte de cache de datos.
 - También se denomina capa de presentación.

2. Arquitectura Cliente/Servidor

Uno de los servicios más populares de Internet es el servicio WWW o web. Existen otros servicios como correo electrónico y mensajería, entre otros, pero la tendencia es hacia su integración con la [WWW](http://www.w3.org/).

Cada servicio tiene su protocolo propio (o protocolos). Así tenemos:

- Web: HTTP
- Correo: SMTP, POP, IMAP
- Mensajería: XMPP y otros
- Intercambio de archivo: FTP, BitTorrent

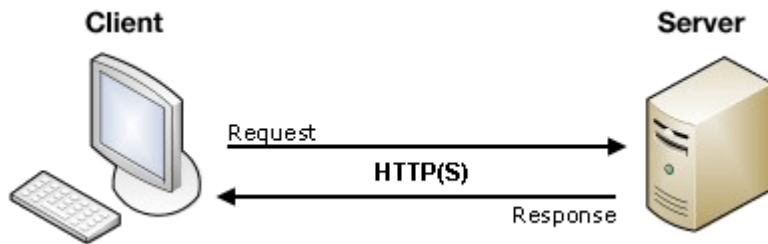
Muchos de los protocolos de Internet, sobre todo los más veteranos, son del tipo **Cliente/Servidor**, frente a algunos más novedosos, como BitTorrent, que son **PeerToPeer (P2P)**.

La principal diferencia entre estos 2 modos es la siguiente:

- Cliente/Servidor: Un dispositivo actúa como Cliente o como Servidor de forma exclusiva.
- PeerToPeer: Un dispositivo actúa como Cliente y como Servidor a la vez.

El protocolo HTTP es un protocolo Cliente/Servidor, en el cual la comunicación sigue el siguiente proceso:

1. El cliente realiza una petición (**request**) al servidor.
2. El servidor devuelve una respuesta (**response**) al cliente.



2.1. Protocolo HTTP/HTTPS

Referencias:

- [Protocolo HTTP - Wikipedia](#)
- [Protocolo HTTP - MDN](#)
- [Protocolo HTTP - CodeAndCoke](#)

El protocolo usado mayoritariamente para la transferencia de información web es el protocolo HTTP (o su versión segura HTTPS). Se trata de un protocolo de texto, en el cual la información entre cliente y servidor se transmite en texto plano.

A continuación, se muestra un ejemplo de una petición y un ejemplo de una respuesta.

Petición del cliente

GET / HTTP/1.1

Host: developer.mozilla.org

Accept-Language: es-ES

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0

Connection: keep-alive

Respuesta del servidor

HTTP/1.1 200 OK

Date: Sat, 09 Oct 2010 14:28:02 GMT

Server: Apache

Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT

ETag: "51142bc1-7449-479b075b2891b"

Accept-Ranges: bytes

Content-Length: 29769

Content-Type: text/html

<!DOCTYPE html... (aquí estarían los 29769 bytes de la página web pedida)

Note

Un cliente web, además de realizar peticiones GET, también puede hacer peticiones de tipo POST. Por ejemplo, al enviar al servidor información de un formulario:

POST /contact_form.php HTTP/1.1

Host: developer.mozilla.org

Content-Length: 64

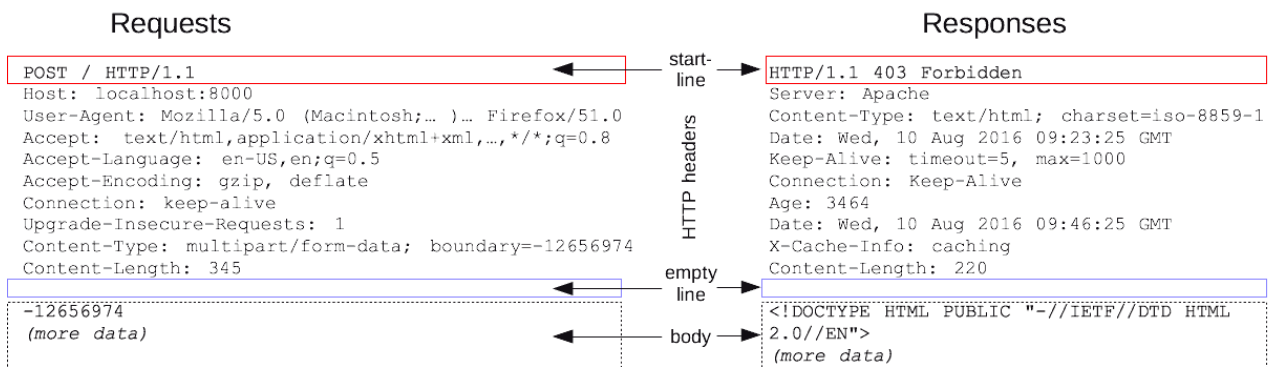
Content-Type: application/x-www-form-urlencoded

name=Juan%20Garcia&request=Envieme%20uno%20de%20sus%20catalogos

Formato de peticiones y respuestas

El formato que siguen tanto las peticiones como las respuestas es el siguiente:

- **Línea inicial**
- **Cabeceras**
- **Línea en blanco**
- **Cuerpo** (opcional en las peticiones)



2.2. Peticiones HTTP y códigos de estado

Existen diversos tipos de [peticiones a un servidor](#), también conocidas como **métodos** o verbos. Algunas de ellas son:

- **GET**
- **POST**
- **PUT**
- **DELETE**
- ...

Los [códigos de estado que devuelve el servidor](#) más habituales son:

- **200 OK**
- 201 Created
- 204 No Content
- 301 Moved Permanently
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- **404 Not Found**
- 500 Internal Server Error

2.3. Clientes web

El cliente web más ampliamente usado es el navegador web o *browser*. Este tipo de software interpreta y renderiza código HTML + CSS. HTML por si sólo permite únicamente realizar peticiones de tipo GET y POST (ésta última mediante el uso de un formulario)

Para realizar todo tipo de peticiones (GET, POST, PUT, DELETE, ...) deberemos hacer uso de Javascript en el navegador.

A continuación se muestra como realizar estas peticiones usando Javascript.

```
fetch('/api/clientes', { method: 'GET' })  
  
  .then ( res => res.json() )  
  
  .then ( data => console.log(data) );
```

```
fetch('/api/clientes/5b4916cb2100bc25330b6ac9', { method: 'GET' })  
  
  .then ( res => res.json())  
  
  .then ( data => console.log(data) );
```

```
fetch('/api/clientes/5b49b5e33808be1b00b982e2', { method: 'DELETE' })  
  
  .then ( res => res.json())  
  
  .then ( data => console.log(data) );
```

```
var cliente = { nombre: "Isabel", apellidos: "López" };
```

```
fetch('/api/clientes', {  
  method: 'POST',  
  body: JSON.stringify(cliente),  
  headers:{  
    'Content-Type': 'application/json'  
  }  
})
```

```
  .then(res => res.json())  
  
  .then(data => console.log(data))
```

```
var cliente = { nombre: "Pepe", apellidos: "Pérez" };
```

```
fetch('/api/clientes/5b4916cb2100bc25330b6ac9', {  
  method: 'PUT',  
  body: JSON.stringify(cliente),  
  headers:{  
    'Content-Type': 'application/json'  
  }  
})
```

```
  .then(res => res.json())  
  
  .then(data => console.log(data));
```

Ni Javascript es el único lenguaje, ni el navegador web es el único entorno, desde el que podemos utilizar los métodos HTTP.

Por ejemplo, podemos realizar peticiones HTTP usando multitud de aplicaciones:

- Clientes para API: Postman, Insomnia, ...
- Aplicaciones de terminal
- Aplicaciones de escritorio
- Otro tipo de aplicaciones

Como ejemplo, se muestra como realizar peticiones HTTP desde la línea de comandos, con la herramienta curl:

GET

```
curl http://localhost:3000/api/articulos
```

```
curl -H 'Content-Type: application/json' \
```

```
-X GET http://localhost:3000/api/articulos
```

POST

```
curl -H 'Content-Type: application/json' \
```

```
-X POST -d '{"nombre": "Botas de invierno","precio": 99.99}' \
```

```
http://localhost:3000/api/articulos
```

PUT

```
curl -H 'Content-Type: application/json' \
```

```
-X PUT -d '{"nombre": "Paraguas","precio": 100.20}' \
```

```
http://localhost:3000/api/articulos/5b609c52c60bd6656205e3d7
```

DELETE

```
curl -H 'Content-Type: application/json' \
```

```
-X DELETE http://localhost:3000/api/articulos/5b609c52c60bd6656205e3d7
```

- Referencia: [Comprobando la API](#)

2.4. Servidores web

Un servidor web es el software encargado de recibir y responder a las peticiones de los clientes.

Según su nivel de complejidad y recursos necesarios, podemos dividirlos en 2 tipos:

- Servidores de **contenido estático**
- Servidores de **contenido dinámico**

Note

Por otro lado, también podemos diferenciar entre:

1. Servidores generalistas: Apache, Tomcat, nginx, IIS.
2. Servidores específicos de aplicación

Los primeros se suelen usar con aplicaciones desarrolladas con lenguajes como PHP, Java, .Net. Los segundos suelen programarse específicamente para implementar el backend de una aplicación web desarrollada para Python, NodeJS.

Tanto unos como otros pueden ofrecer tanto contenido estático como computo.

2.4.1. Servidores de contenido estático

Estos servidores web son funcionalmente muy sencillos. Se limitan a atender las peticiones de los clientes y a servir el contenido estático solicitado por ellos: HTML, CSS y Javascript para ejecutar en el lado cliente. También sirven imágenes, fuentes, ...

Suelen ser muy baratos e incluso gratuitos muchas veces. Se usan para desplegar sitios web y aplicaciones sencillas (del lado cliente).

Los servidores de computo, por otro lado, ofrecen la posibilidad de ejecutar código en el propio servidor, por lo cual suponen un mayor costo al proveedor tanto energético como en tiempo. Su complejidad y gestión también es menos simple. Aunque es posible encontrar algún que otro proveedor con un *free tier*, es habitual la necesidad de realizar un pago mensual.

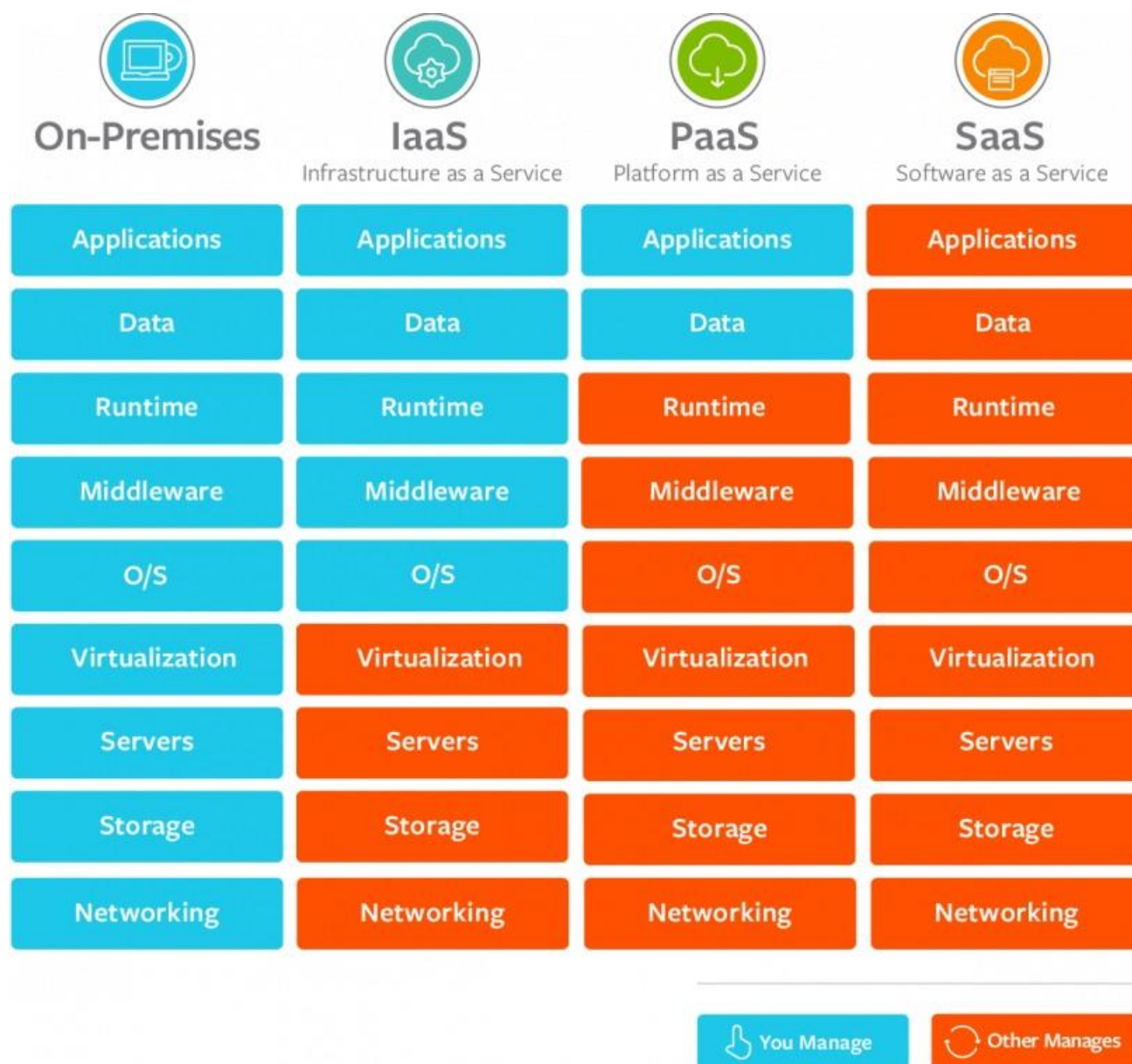
2.4.2. Servidores de contenido dinámico

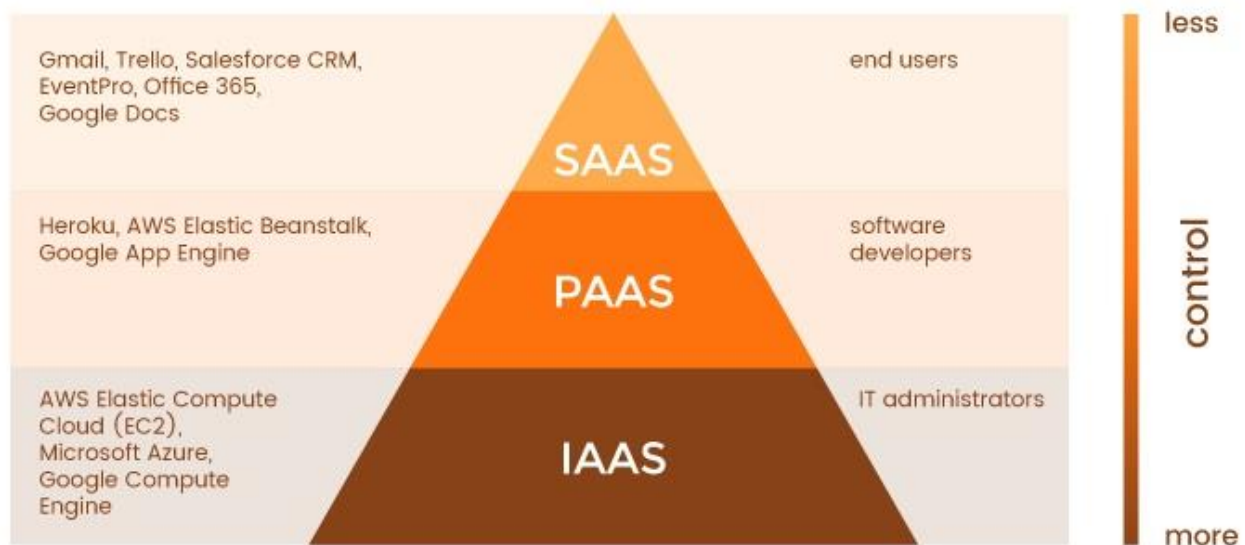
Son aquellos servidores que, antes de servir el contenido, realizan algún tipo de computo. Suele ser habitual la búsqueda y modificación de información en bases de datos y el posterior renderizado y envío de vistas al cliente (la mayoría de las veces es un navegador web). Otras operaciones que se realizan en el lado servidor suelen ser la gestión de la autenticación y autorización.

Los lenguajes más utilizados para estos fines, sin ningún orden en particular, son:

- PHP
- Java / JSP
- Python
- NodeJS
- ...

Los servidores de contenido dinámico se pueden clasificar en diferentes categorías. Las más habituales son las siguientes:





Note

También existe **DBaaS** (Base de datos como servicio). Se puede considerar equivalente al PaaS, pero aplicado a bases de datos.

Algunos proveedores de estos servicios son:

- [IaaS \(Infraestructura como servicio\)](#)
 - Amazon: mediante EC2 (Elastic Compute Cloud)
 - Digital Ocean: mediante Droplets
- [PaaS \(Plataforma como servicio\)](#)
 - Heroku
 - Vercel

Una variante de los servidores de contenido dinámico son los [serverless](#)

3. Tecnologías para el backend

3.1. Lenguajes del lado del servidor



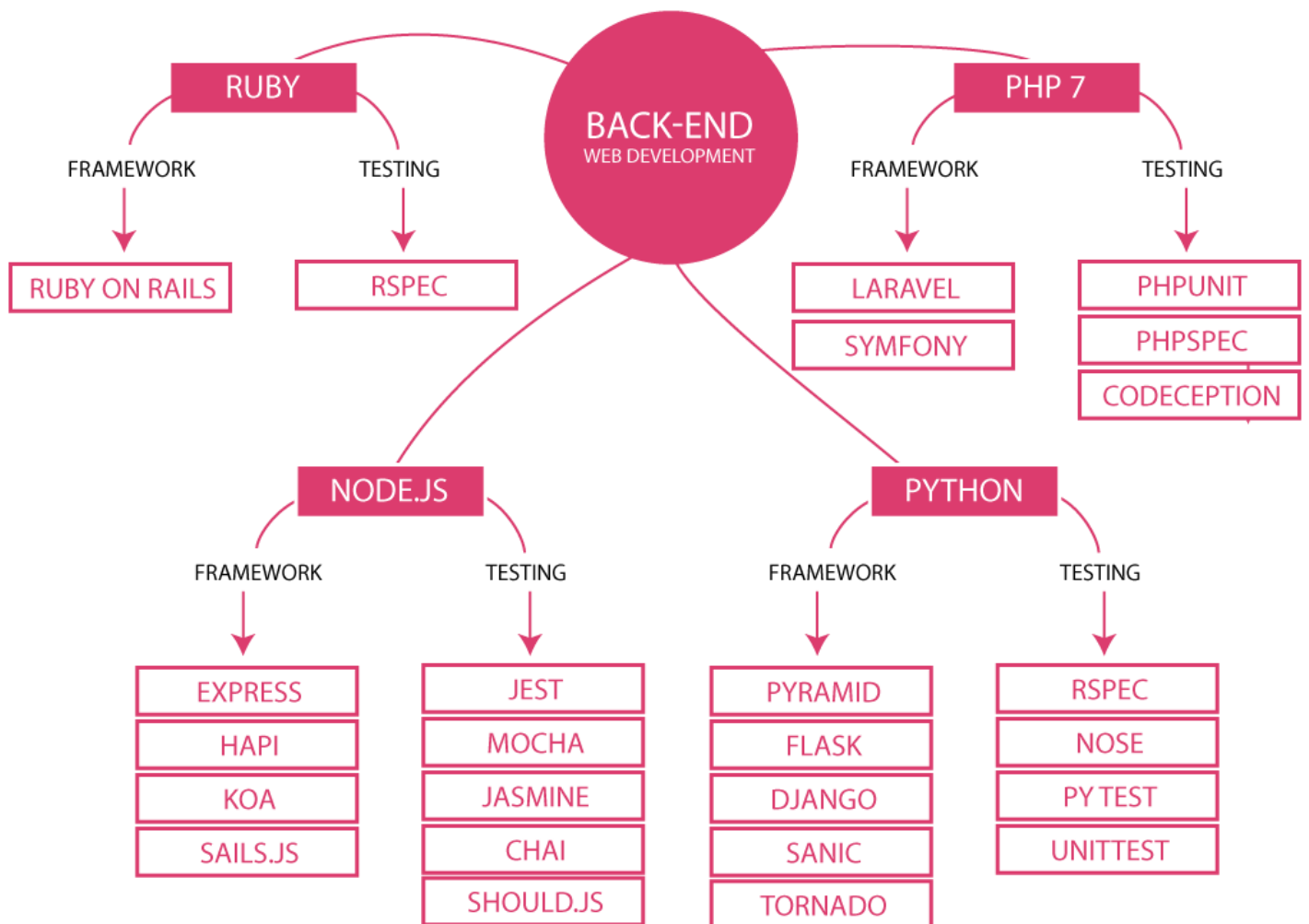
- **PHP:** PHP Hypertext Preprocessor. Uno de los lenguajes más utilizados para la creación de todo tipo de CMS.
- **JSP:** JavaServer Pages. Es la propuesta de Java similar a PHP.
- **ASP:** Active Server Pages. Es la propuesta de Microsoft similar a PHP.
- **Java:** es un lenguaje multiplataforma propiedad de Oracle.
- **C#:** es un lenguaje multiplataforma propiedad de Microsoft.
- **Ruby:** es un lenguaje de programación interpretado, reflexivo y orientado a objetos.
- **Python:** es un lenguaje de programación multiparadigma que hace hincapié en el código legible.
- **Javascript (NodeJS):** cada vez más popular puesto que se usa también en el lado cliente.

3.2. Frameworks del lado servidor



Framework	Lenguaje
Laravel	PHP
Spring	Java
.NET	C#
Ruby on rails	Ruby
Django	Python
Express	Javascript

3.3. Resumen

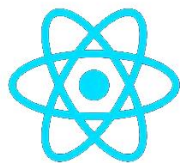


4. Tecnologías para el frontend

4.1. Lenguajes del lado del cliente

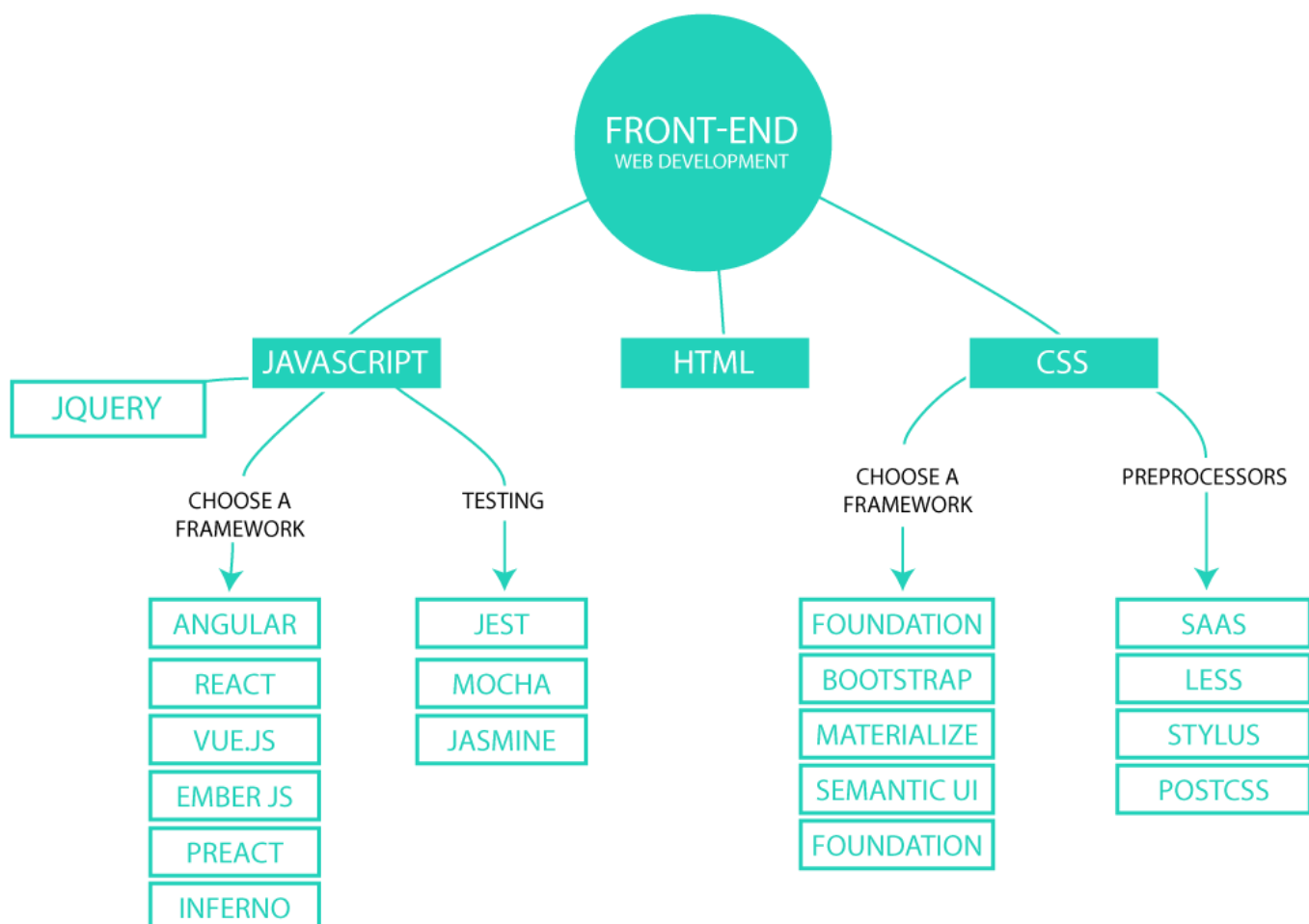


4.2. Frameworks del lado cliente



- **Angular**
- **React**
- **Vue**
- **Svelte** (compilador)

4.3. Resumen



5. Tecnologías Javascript Fullstack

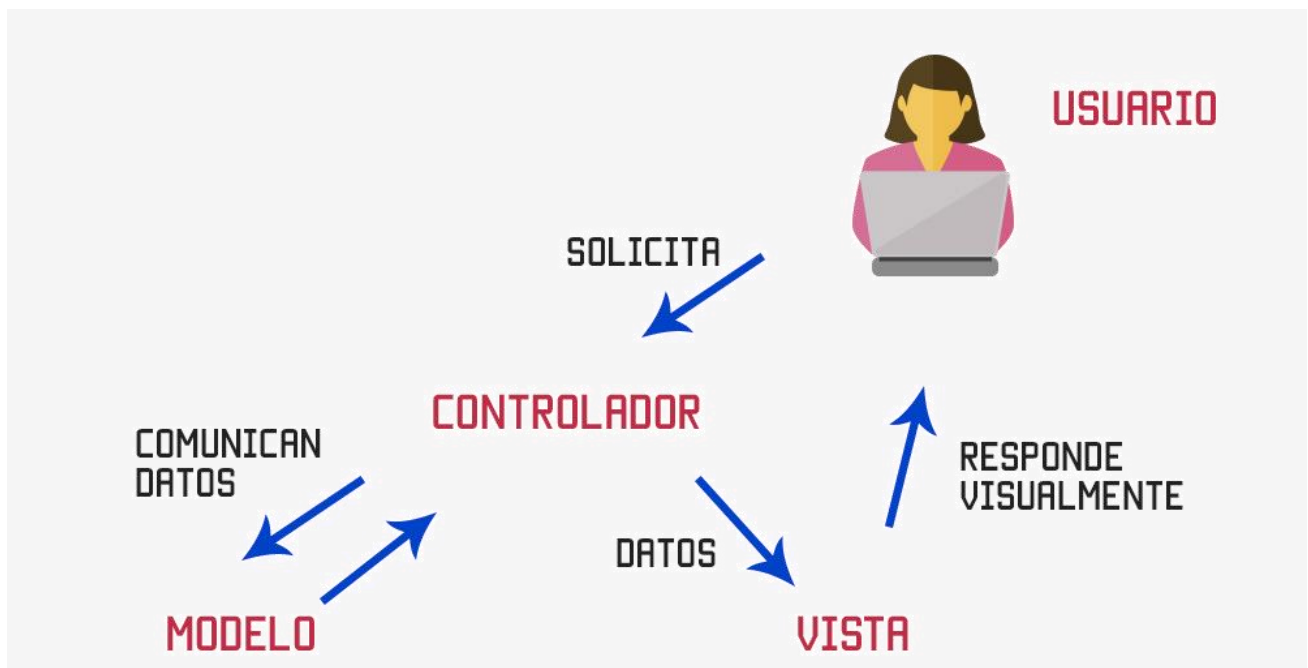
5.1. Frameworks para servidor y cliente



- Universal
- Next
- Nuxt
- SvelteKit

6. MVC o SPA

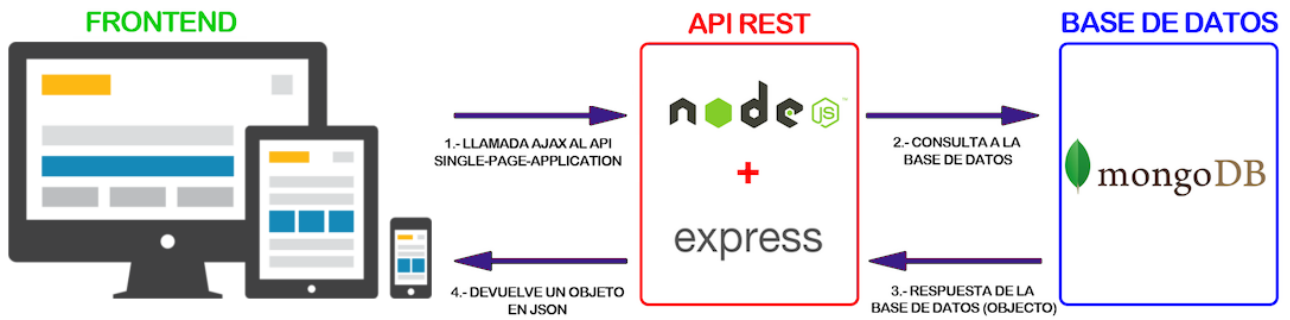
6.1. Modelo-Vista-Controlador



- patrón de diseño arquitectónico
- se ejecuta casi toda la lógica de aplicación en el backend.
- el servidor sirve múltiples vistas, cada vista es una página HTML.
- es una aplicación de múltiples páginas (MPA).

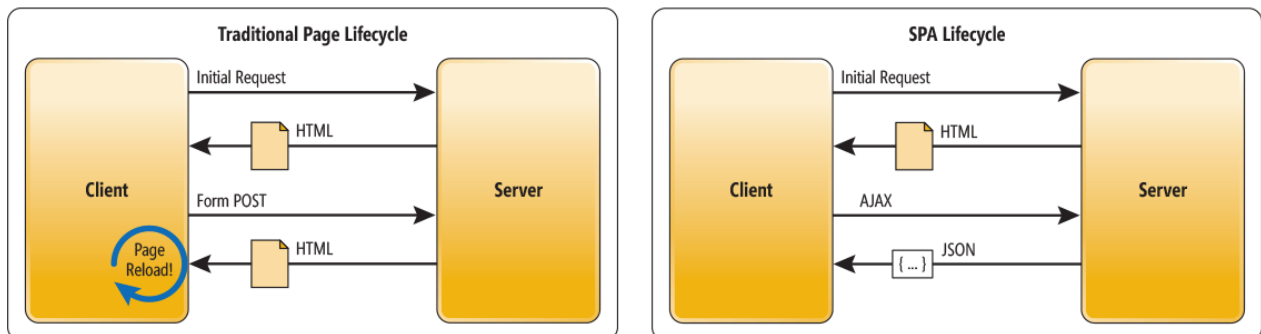
<https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador>

6.2. Single Page Application

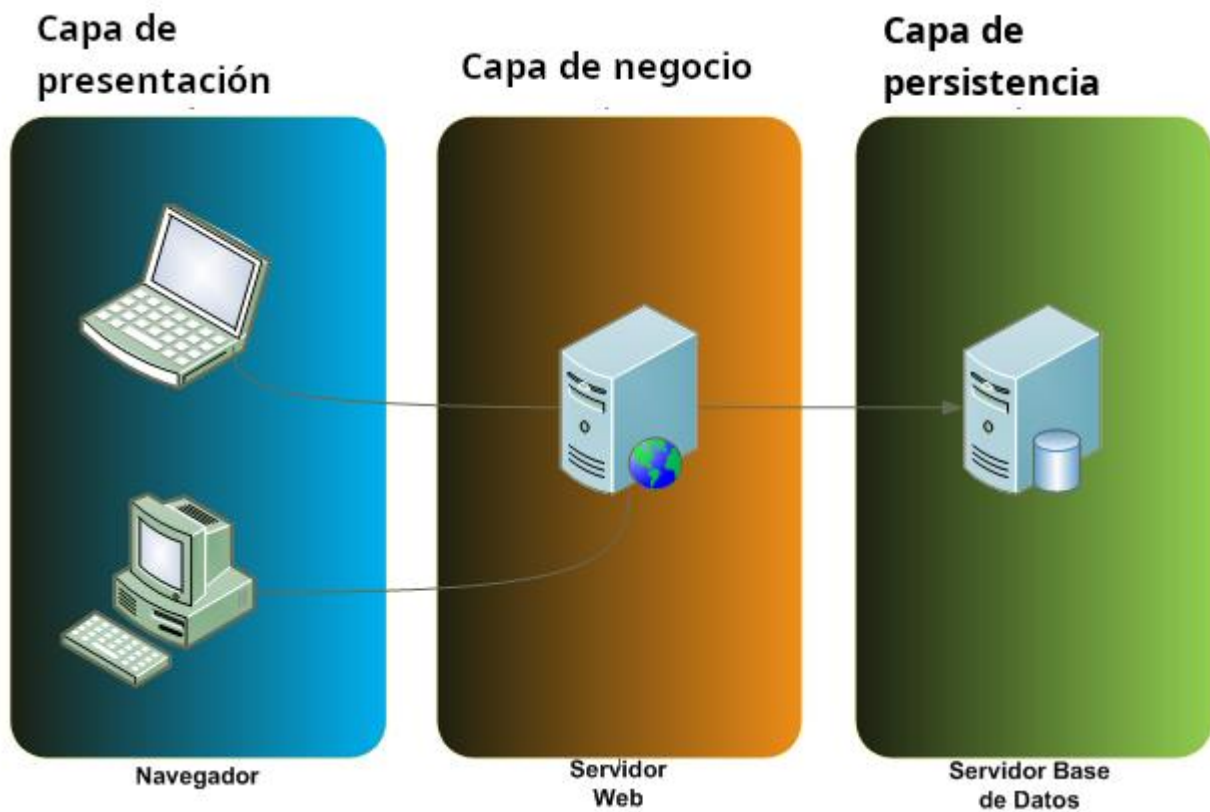


- se ejecuta gran parte de la lógica de la aplicación en el frontend.
- el servidor sirve datos mediante una API web.
- principalmente se usan 2 formatos para el intercambio de datos:
 - XML
 - JSON

6.3. MVC vs SPA



6.4. Arquitectura de 3 capas



7. Persistencia de los datos

- Uso de archivos
- Bases de datos relacionales
 - Subtipo importante: **BBDD objeto-relacionales**.
- Bases de datos no relacionales
 - Subtipo importante: **BBDD noSQL**.

7.1. Bases de datos relacionales



7.2. Bases de datos no relacionales (noSQL)



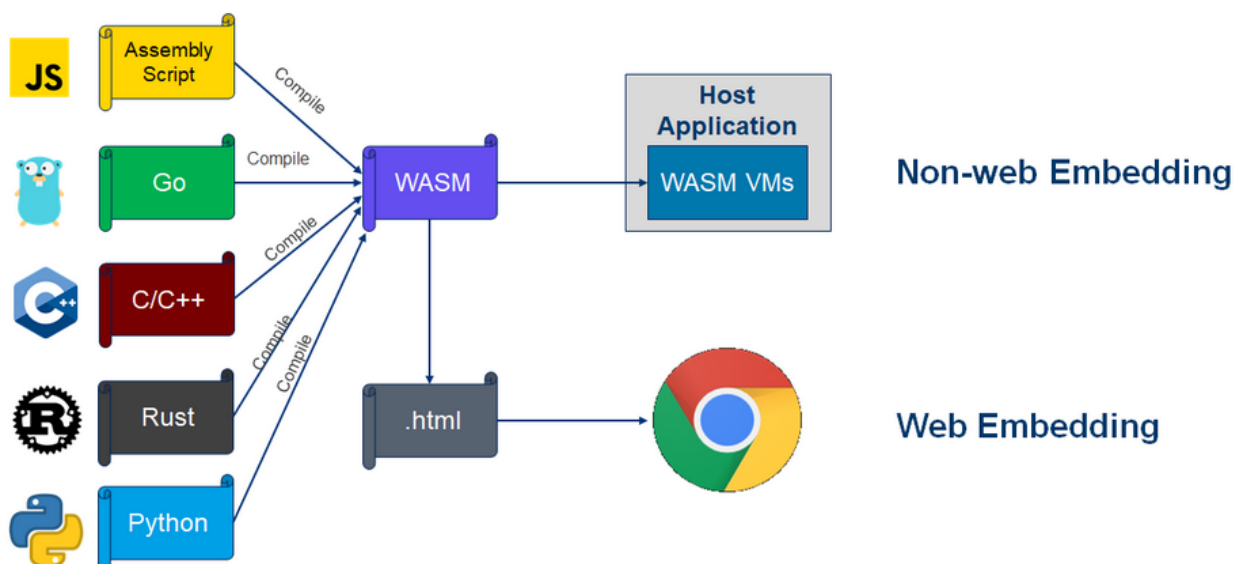
8. Viajando al futuro, que ya es presente



- [WebAssembly](#)

8.1. WebAssembly (Wasm)

- Formato binario pequeño y rápido que promete un rendimiento casi nativo para las aplicaciones web.
- Los principales navegadores son compatibles con WebAssembly.
- Los desarrolladores escriben en el lenguaje de su elección (C, C++, ...), que luego se compila en bytecode WebAssembly.
- Para casos de uso intensivo de rendimiento, como juegos, transmisión de música, edición de vídeo y aplicaciones CAD.



- [WebAssembly explicado](#)